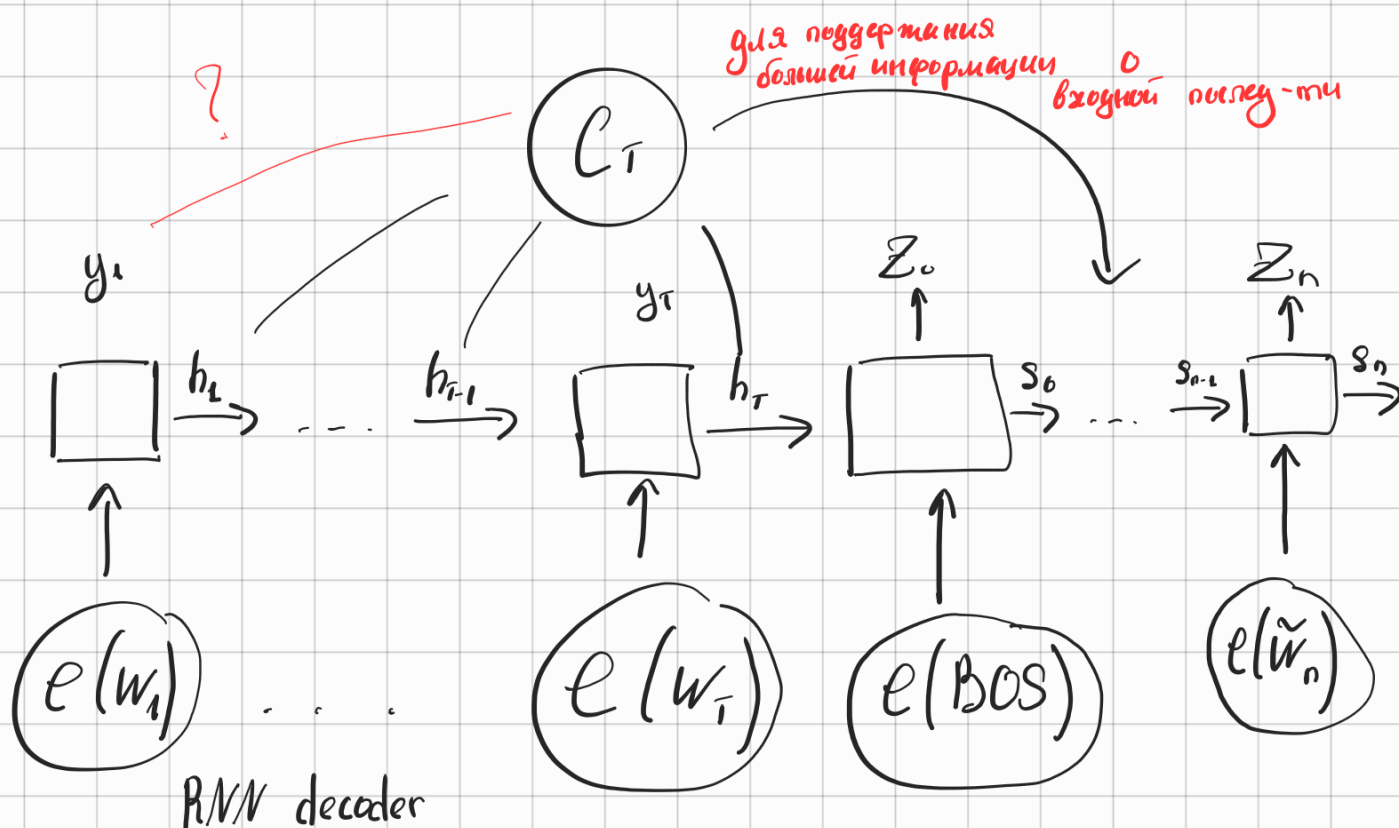


Лекция 8. Трансформеры

RNN



$$s_t = g(W_1 x_t + W_2 s_{t-1} + Wc_t)$$

Цель трансформера - избавиться от рекурр. связей (s_t) и сфокусироваться на работе этой операции агрегации контекста

RNN encoder

$$\text{Вход: } E = [e(w_1), \dots, e(w_r)]$$

$$\text{Вход: } H = [h_1, \dots, h_r]$$

$$h_0 := 0$$

для $t := 1$ до T :

$$h_t, y_t = \text{RNNCell}(h_{t-1}, e(w_t))$$

RNN decoder

$$\text{Вход: } \tilde{E} = [e(\text{Bos}), \dots, e(\tilde{w}_n)]$$

$$H = [h_1, \dots, h_r]$$

$$\text{Вход: } Z = [z_1, \dots, z_n]$$

для $t := 0$ до n : $s_0 := \text{вход из BOS}$

$$c_t := \langle \text{по алгоритму Attention} \rangle$$

$$s_t, z_t = \text{RNNCell}(s_{t-1}, e(\tilde{w}_t))$$

Attention(s_{t-1}, H, H)

Attention

$$\text{ВХОД: } q \in \mathbb{R}^d \text{ [например } s_{t-1}]$$

запрос

$$K \in \mathbb{R}^{T \times d} [H]$$

ключи

$$V \in \mathbb{R}^{T \times d} [Y]$$

значения

(в ситуациях рассматрив. ранее это также H)

Интерпретация: S — запрос, с которого обращаемся в базу знаний K, V . $h_1, h_2, \dots$ — выступают как ключ, тем, даёт информацию, насколько запрос совпадает с ключом. На основе этой близости достаем значение y , конкретно в рассматриваемом случае это также h .

$$\text{Выход: } c \in \mathbb{R}^d$$

$$c_t = \sum_{i=1}^T y_i d_{it}$$

$$d_{it} = \frac{\exp(\text{sim}(h_i, s_{t-1}))}{\sum_{j=1}^T \exp(\text{sim}(h_j, s_{t-1}))}$$

$$d_{it} = \frac{\exp(\text{sim}(h_i, s_{t-1}))}{\sum_{j=1}^T \exp(\text{sim}(h_j, s_{t-1}))}$$

Attention скро

берем соответствующую i -ю близость:

$$\text{sim}(x, y) = \langle x, y \rangle$$

$$1. m := Kq$$

$$2. d_i := \text{Softmax}(m)$$

далее складываем все строки V с весами d

$$3. c_i := \frac{V^T d}{(d \times 1)^T (1 \times 1)}$$

Как оставить лишь Attention?

RNN encoder

$$\text{Вход: } E = [e(w_1), \dots, e(w_T)]$$

$$\text{Выход: } Y = [y_1, \dots, y_T]$$

для $t := 1$ до T :

$$c_t := \text{Attention}(e(w_t), E, E)$$

то есть теперь учитываются все вход на каждой итерации

$$y_t := FC(c_t)$$

RNN decoder

$$\text{Вход: } \tilde{E} = [e(\text{Bos}), \dots, e(\tilde{w}_n)]$$

ситуация обучения, уже знаем все следующие токены

$$\text{Выход: } Z = [z_1, \dots, z_n]$$

контекст новых токенов

$$s_t := \text{Attention}(e(\tilde{w}_t), \tilde{E}, \tilde{E})$$

$$c_t := \text{Attention}(s_t, Y, Y)$$

контекст токенов исходной посл-ти

$$z_t := FC(c_t)$$

Теперь:

- каждое слово в посл-ти (кодировщика или декодировщика) имеет непосредственную связь со всеми другими словами в этой посл-ти
- Т.е. посл-ть „смотрит“ на всю себя, откуда и название алгоритма

Как можно

заместить, почти все операции независимы в вычислениях

Attention

Вход: $Q \in \mathbb{R}^{T \times d}$

$K \in \mathbb{R}^{T \times d}$ [K]

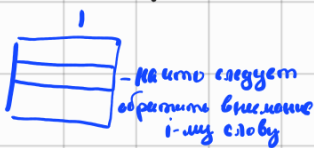
$V \in \mathbb{R}^{T \times d}$ [V]

Выход: $C \in \mathbb{R}^d$

1. $m := Q K^T \in \mathbb{R}^{T \times T}$

2. $\alpha := \text{Softmax}(m)$ [dim=1]

3. $C_i := \alpha V$

 - на что следует обратить внимание i-ую строку

Attention(Q, x, v) =

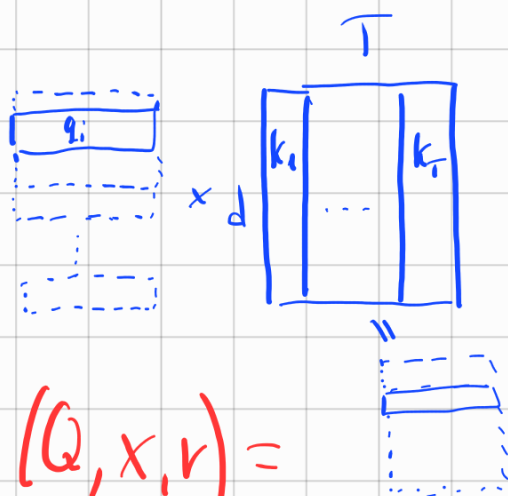
$\text{Softmax}(Q K^T) [\text{dim}=1] \cdot V$

$$C_t = \frac{\sum_{i=1}^T y_i d_{i,t}}{\sum_{j=1}^T \exp(\text{sim}(h_j, s_{t-1}))}$$

$$d_{i,t} = \frac{\exp(\text{sim}(h_i, s_{t-1}))}{\sum_{j=1}^T \exp(\text{sim}(h_j, s_{t-1}))}$$

Attention score

для каждой i-й суммы: $\text{sim}(x, y) = \langle x, y \rangle$



RNN encoder

Вход: $E = [e(w_1), \dots, e(w_r)]$

Вход: $Y = [y_1, \dots, y_r]$

1. $C := \text{Attention}(E, E, E)$

2. $Y := \text{FC}(C)$

RNN decoder

Вход: $\tilde{E} = [e(\text{Bos}), \dots, e(\tilde{w}_n)]$

$Y = [y_1, \dots, y_r]$
Вход: $Z = [z_1, \dots, z_r]$

1. $S := \text{Attention}(\tilde{E}, \tilde{E}, \tilde{E})$

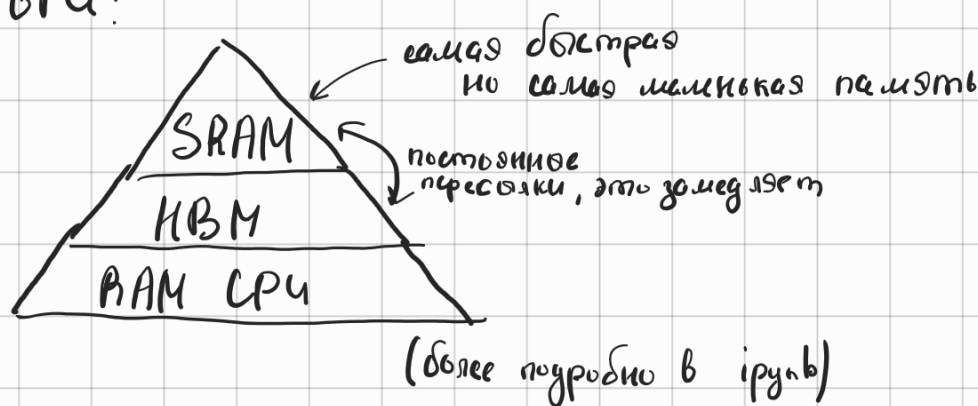
2. $C := \text{Attention}(S, Y, Y)$

3. $Z := \text{FC}(C)$

Latency vs Throughput
(для CPU) (для GPU)

low-latency - CPU
high-throughput - GPU

В GPU:



Обучаемый Attention

1. $\text{Attention}(QW_q, KW_k, VW_v)$

$d \times d$ $d \times d$ $d \times d$

$\text{Att} \dots (E, E, E)$ - так применение возможно не имеет смысла

Давайте сначала переведем в другое пространство, "выделим" нужную информацию для ключей, запросов и значений, да и в целом сделаем Attention обучаемым

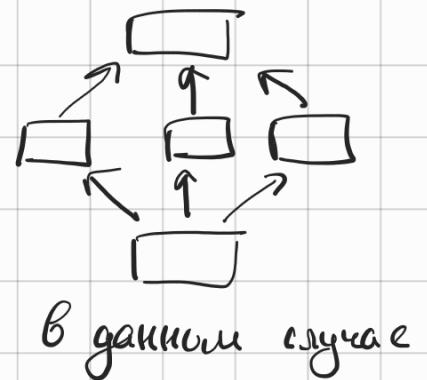
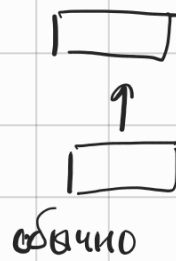
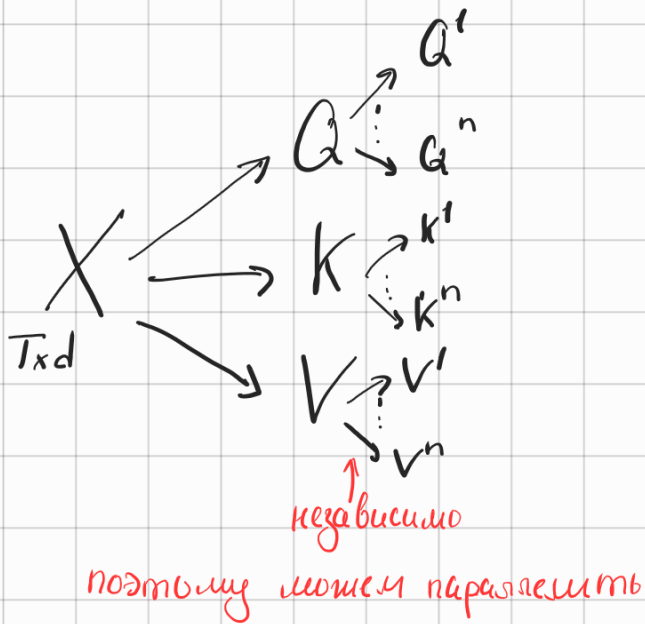
2. Многоруковость (multi-head attention)

$$\text{head}_i = \text{Attention}\left(\underbrace{QW_q^i}_{d \times \frac{d}{n}}, \underbrace{KW_k^i}_{d \times \frac{d}{n}}, \underbrace{VW_v^i}_{d \times \frac{d}{n}}\right), \quad i = \overline{1, n}$$

$$\text{Multihead Attention}(Q, K, V) = \text{concatenate}(\text{head}_1, \dots, \text{head}_n) \underbrace{W_o}_{d \times d}$$

$\text{Attention}(E, E, E)$ - самовнимание
 $\text{Attention}(S, Y, Y)$ - crossattention

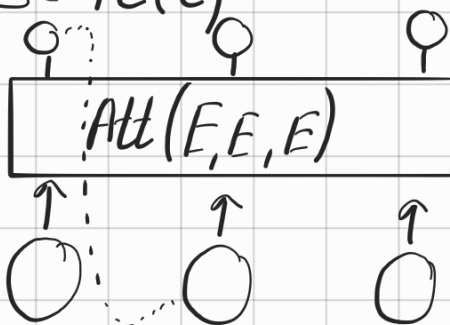
Das Self-attention:



Смысл: получили n разных сетей, которые могут улавливать разные зависимости.

В декодере:

1. $S = \text{Attention}(\tilde{E}, \tilde{E}, \tilde{E})$
2. $C = \text{Attention}(S, Y, Y)$
3. $Z = \text{FC}(C)$



в этой репрезентации
 не должны быть информации о будущем
 контексте, и производит уточно шток

Поэтому применяют маскированную Attention

$$1. \quad m := QK^T + M, \quad M = \begin{pmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{pmatrix}$$

$$2. \alpha := \text{Softmax}(m) \{ \dim = 1 \}, \text{softmax } c_m \rightarrow \infty = 0$$

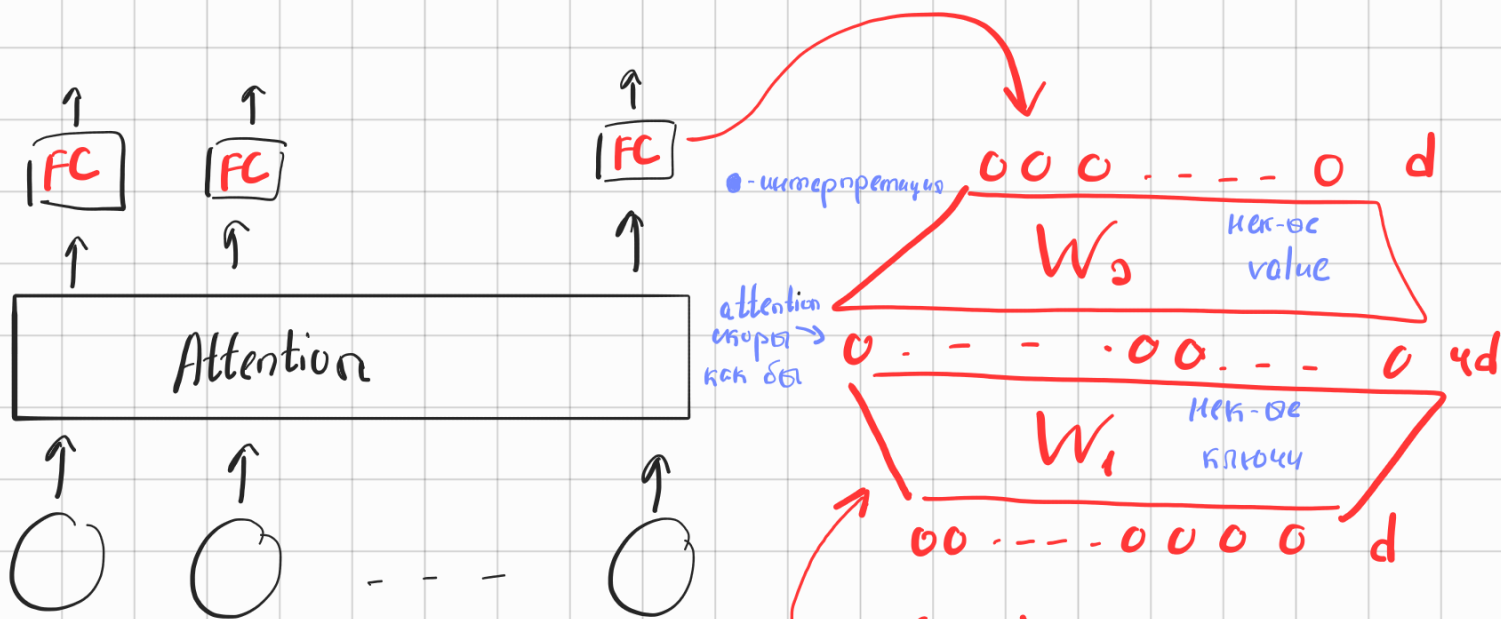
3. $C := \alpha V \rightarrow$

↑ Masked Attention ↑

решили проблему утечки меток

1. $S = \text{Attention}(\tilde{E}, \tilde{E}, \tilde{E})$
2. $C = \text{Attention}(S, y, y)$
3. $Z = FC(c)$

Как работает FC



$$Z = g(xw_1)w_2$$

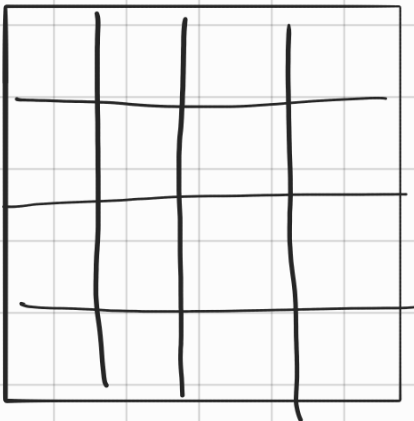
encoder (то есть только
символизация)

• FC - самые тяжёлые блоки, наибольшее кол-во параметров, хранится наибольшая часть знаний трансформера как языковой модели о внешнем мире

Вычисления в Attention блоках значительно меньше

• Многочислен ^{по той же причине, что и RNN} неводно из-за отсутствия возможности распараллелить

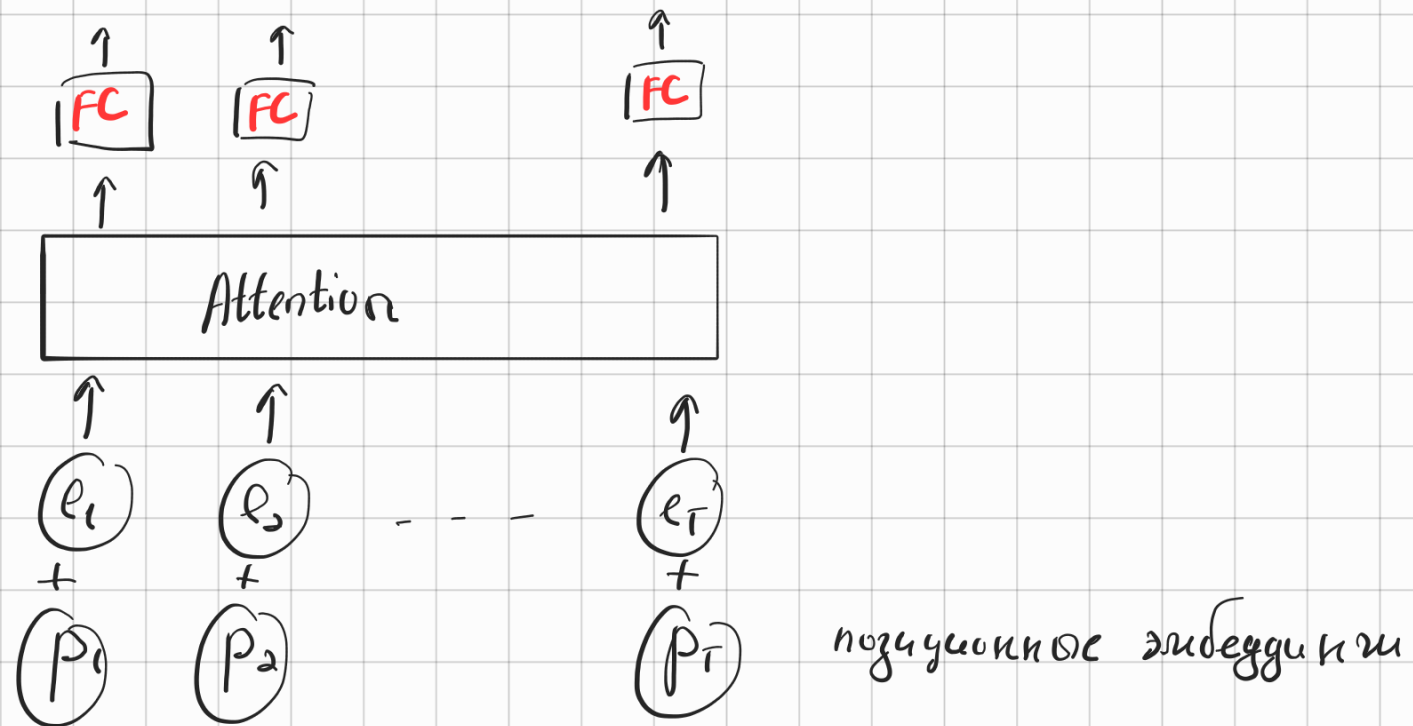
Трансформеры применяются и в других смежных областях. Например изображения:



→ вытянуть в одну строку, послед-ть
Оказ-ся трансф-ры работают
лучше свёрточным зрением

Position Encoding

Добавляем информацию о порядке



$P \in \mathbb{R}^{T \times d} \rightarrow$ можно также сделать обучаемыми

синусоидная кодировка.

$$P_{m,2i} = \sin\left(\frac{m}{10000^{2i/d}}\right)$$

m-номер строки

$$P_{m,2i+1} = \cos\left(\frac{m}{10000^{2i/d}}\right)$$

$$i := 1, \frac{d}{2}$$

Пример:

$$d=3$$

p_1	000
p_2	001
$\vdots$	010
	011
	100
	$\vdots$

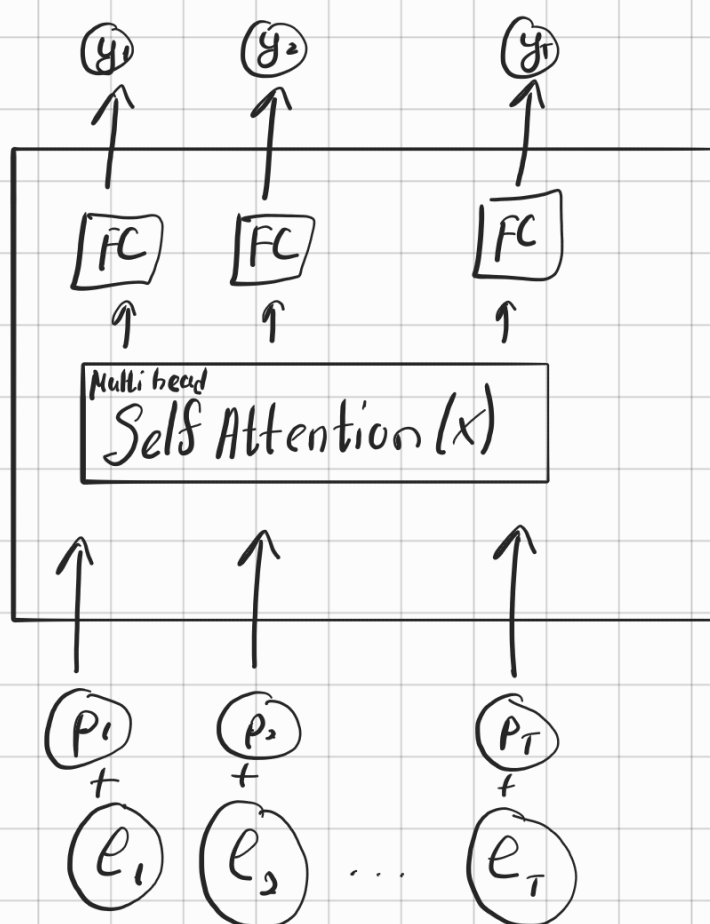
закодировали позицию в виде вектора.
но влюб это не работает

- $\sin(\varphi + \nu t)$

частота (ν) определяется i (читаем t за аргумент)

И таким образом, то что написали, это на самом деле аналог

- сеть обозначение через линейную алгебру



т.к. на входе так же сохраняется размерность d , можем накинуть L слоев внимания

